

Programación (01/04/2026)

EXAMEN RECUPERACIÓN – TEMAS 1-7

1. (2p) Dada la siguiente clase que utiliza anotaciones de *Lombok*:

```
@NoArgsConstructor
@AllArgsConstructor
@Getter
class Alumno {
    int nia;
    @Setter
    String nombre;

    public static void obtenerDescripcion(){
        System.out.println("Alumno del IES MUTXAMEL");
    }
}
```

a) (1,5p) Transcribe la estructura de la clase equivalente que se generaría al compilarla. ¿Se cumple el principio de encapsulamiento? Justifica tu respuesta.

b) (0,5p) Si quisiéramos ejecutar el método *obtenerDescripcion()* desde una clase externa, ¿cómo se realizaría la llamada?

2. (1,25p) Observa el siguiente fragmento de código en un *main* cualquiera:

```
String informe = Procesador.analizar("Error de sistema");
int codigo = Procesador.analizar(404);
boolean esCritico = Procesador.analizar(9.5, 5.0);
```

a) (0,75p) Escribe las cabeceras de los métodos originales que permiten que esto funcione.

b) (0,5p) ¿Qué pasaría si intentamos crear un cuarto método con la cabecera *public static void analizar(int n)*? Explícalo con tus palabras y menciona el concepto de programación que se está aplicando.

3. (1p) Realiza la traza detallada del siguiente programa:

```
int resultado = 0;

for (int j = 1; j <= 3; j++) {
    for (int i = 3; i >= 1; i--) {
        if ((i + j) % 2 == 0) {
            resultado *= 2;
        } else if (i == j) {
            resultado -= 2;
        } else {
            resultado += 1;
        }
    }
}
```

4. (1,25p) Dado el siguiente método recursivo:

```
public static int misterio(int x) {
    if (x != 0) {
        return (x % 2) + misterio(x - 1);
    } else {
        return 1;
    }
}
```

indica qué valor devuelve si se le llama de la siguiente manera:

```
int resultado = misterio(4);
```

5. (1p) Estás desarrollando una *app* para un dispositivo multifunción. Tienes las siguientes dos interfaces:

```
public interface Reproductor {      public interface Camara {
    void reproducirAudio();          void hacerFoto();
}
```

- a) (0,5p) Escribe la cabecera de una clase llamada *SmartPhone* que deba cumplir obligatoriamente con los contratos de ambas interfaces. ¿Qué concepto de programación estás aplicando? Explícalo.
- b) (0,5p) Ahora, necesitas que *SmartPhone* use también una interfaz llamada *LocalizadorGPS*, que tiene el método abstracto *obtenerCoordenadas()*. Si no quieres crear una nueva clase *.java* completa sólo para realizar una prueba rápida, ¿qué solución temporal podrías implementar dentro del programa principal para poder ejecutarlo?
6. (1,25p) Si tienes una colección cualquiera de objetos *Persona* (**nombre** y **edad**) y requieres ordenarlos por dos criterios distintos (por ejemplo, primero por *edad* y luego por *nombre*). ¿Qué opción/es tenemos en *Java* para lograrlo? Explica cómo funcionan e implementa el esqueleto de ordenación necesario para uno de los dos criterios.
7. (1p) Estás gestionando una lista de reproducción en una *app*: ["Canción 1", "Canción 2", "Canción 3"]. Elige la estructura correcta y escribe el código correspondiente para realizar las siguientes tareas:
- a) (0,5p) Recorre la lista hasta situarte justo después de la "Canción 2" e inserta una nueva canción llamada "Publicidad" en esa posición exacta.
- b) (0,5p) Retrocede en la lista para mostrar por pantalla la canción que acabas de introducir.
8. (1,25p) Estás programando el software para un peaje de autopista. Los coches llegan y son atendidos estrictamente en el orden en que aparecieron. Tienes una estructura que almacena las matrículas de los coches en espera: ["1234-ABC", "5678-DEF", "9012-GHI"]
- a) (0,357p) ¿En qué consiste el principio *FIFO* que rige esta estructura? Menciona el nombre de la interfaz y la clase de *Java* que se suele utilizar para implementarla.
- b) (0,5p) Indica qué métodos usarías para las siguientes acciones:
- Llega un nuevo coche ("3344-JKL") al final de la cola.
 - El primer coche de la fila paga y abandona el peaje.
 - Consultar cuál es el siguiente coche que va a pagar.
- c) (0,357p) A veces, un vehículo de emergencias (como una ambulancia) necesita colarse porque siempre tiene prioridad. ¿Existe alguna estructura más eficiente para gestionar este tipo de vehículos? Indica qué método/s usarías en estos casos.